

Bayesian Inference Engine Visualiser

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Bayesian Confidence Engine</title>
<style>
@import url('https://fonts.googleapis.com/css2?family=Courier+Prime:ital,wght@0,400;0,700;1,400&family=Bebas+Neue&display=swap');

:root {
  --bg: #0a0a0f;
  --panel: #0f0f1a;
  --border: #1e1e3a;
  --human: #e85d4a;
  --ai: #4ae8b0;
  --neutral: #888;
  --text: #c8c8d4;
  --dim: #555;
  --gold: #f0c060;
  --warn: #e8954a;
}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  background: var(--bg);
  color: var(--text);
  font-family: 'Courier Prime', monospace;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 20px;
  background-image:
    radial-gradient(ellipse at 20% 0%, rgba(74,232,176,0.04) 0%, transparent 50%),
    radial-gradient(ellipse at 80% 100%, rgba(232,93,74,0.04) 0%, transparent 50%);
}

h1 {
  font-family: 'Bebas Neue', sans-serif;
  font-size: clamp(2rem, 5vw, 3.5rem);
  letter-spacing: 0.12em;
  color: var(--text);
  text-align: center;
  margin-bottom: 4px;
}

.subtitle {
  font-size: 0.75rem;
  color: var(--dim);
  letter-spacing: 0.2em;
  text-transform: uppercase;
  text-align: center;
  margin-bottom: 30px;
}

.engine {
  width: 100%;
  max-width: 900px;
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

.panel {
  background: var(--panel);
  border: 1px solid var(--border);
  padding: 20px;
  position: relative;
}

.panel-label {
  font-family: 'Bebas Neue', sans-serif;
  font-size: 0.85rem;
  letter-spacing: 0.2em;
  color: var(--dim);
  margin-bottom: 14px;
  border-bottom: 1px solid var(--border);
  padding-bottom: 8px;
}

/* — BELIEF METER — */
.belief-section {
  grid-column: 1 / -1;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.duel {
  display: flex;
  align-items: center;
  gap: 12px;
}

.agent-label {
  font-family: 'Bebas Neue', sans-serif;
  letter-spacing: 0.12em;
  width: 80px;
  font-size: 1.1rem;
}
```

```
.agent-label.human { color: var(--human); text-align: right; }
.agent-label.ai { color: var(--ai); }
```

```
.bar-track {
  flex: 1;
  height: 28px;
  background: #111;
  border: 1px solid var(--border);
  position: relative;
  overflow: hidden;
}
```

```
.bar-fill {
  height: 100%;
  transition: width 0.7s cubic-bezier(0.23, 1, 0.32, 1);
  position: relative;
}
```

```
.bar-fill.human-fill {
  background: linear-gradient(90deg, #6a1a10, var(--human));
  float: right;
}
```

```
.bar-fill.ai-fill {
  background: linear-gradient(90deg, var(--ai), #10604a);
}
```

```
.bar-pct {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  font-size: 0.8rem;
  font-weight: 700;
  pointer-events: none;
}
```

```
.human-fill .bar-pct { left: 8px; color: #fff; }
.ai-fill .bar-pct { right: 8px; color: #000; }
```

```
.prior-display {
  text-align: center;
  font-size: 0.7rem;
  color: var(--dim);
  letter-spacing: 0.1em;
}
```

```
/* —— SCENARIO PICKER —— */
.scenario-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 8px;
}
```

```
.scenario-btn {
  background: transparent;
  border: 1px solid var(--border);
  color: var(--text);
  font-family: 'Courier Prime', monospace;
  font-size: 0.72rem;
  padding: 10px 10px;
  cursor: pointer;
  text-align: left;
  transition: all 0.2s;
  line-height: 1.4;
}
```

```
.scenario-btn:hover {
  border-color: var(--gold);
  color: var(--gold);
  background: rgba(240,192,96,0.05);
}
```

```
.scenario-btn.selected {
  border-color: var(--ai);
  color: var(--ai);
  background: rgba(74,232,176,0.07);
}
```

```
.scenario-btn .tag {
  display: block;
  font-size: 0.6rem;
  color: var(--dim);
  letter-spacing: 0.15em;
  margin-bottom: 3px;
  text-transform: uppercase;
}
```

```
/* —— OUTCOME PANEL —— */
.outcome-grid {
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

```
.outcome-row {
  display: flex;
  align-items: center;
  gap: 10px;
}
```

```
.outcome-name {
  font-size: 0.72rem;
}
```

```
width: 160px;
color: var(--text);
flex-shrink: 0;
}

.outcome-bar-track {
flex: 1;
height: 14px;
background: #111;
border: 1px solid var(--border);
overflow: hidden;
}

.outcome-bar-fill {
height: 100%;
transition: width 0.7s cubic-bezier(0.23, 1, 0.32, 1);
}

.outcome-pct {
font-size: 0.7rem;
width: 38px;
text-align: right;
color: var(--gold);
}

/* — LOG — */
.log-section {
grid-column: 1 / -1;
}

.log-body {
height: 140px;
overflow-y: auto;
font-size: 0.72rem;
line-height: 1.7;
color: var(--dim);
padding-right: 4px;
}

.log-body::-webkit-scrollbar { width: 3px; }
.log-body::-webkit-scrollbar-thumb { background: var(--border); }

.log-entry { border-bottom: 1px solid #111; padding: 3px 0; }
.log-entry .ts { color: #333; margin-right: 8px; }
.log-entry .ev-human { color: var(--human); }
.log-entry .ev-ai { color: var(--ai); }
.log-entry .ev-sys { color: var(--gold); }
.log-entry .ev-warn { color: var(--warn); }

/* — CONTROLS — */
.controls {
grid-column: 1 / -1;
display: flex;
gap: 10px;
flex-wrap: wrap;
}

.btn {
font-family: 'Bebas Neue', sans-serif;
letter-spacing: 0.15em;
font-size: 0.9rem;
padding: 10px 22px;
border: 1px solid;
cursor: pointer;
background: transparent;
transition: all 0.2s;
flex: 1;
min-width: 120px;
}

.btn-primary { border-color: var(--ai); color: var(--ai); }
.btn-human { border-color: var(--human); color: var(--human); }
.btn-reset { border-color: var(--dim); color: var(--dim); }

.btn-primary:hover { background: rgba(74,232,176,0.1); }
.btn-human:hover { background: rgba(232,93,74,0.1); }
.btn-reset:hover { background: rgba(136,136,136,0.1); }

/* — STATS — */
.stats-row {
grid-column: 1 / -1;
display: flex;
gap: 16px;
flex-wrap: wrap;
}

.stat-box {
flex: 1;
min-width: 100px;
background: var(--panel);
border: 1px solid var(--border);
padding: 12px 14px;
text-align: center;
}

.stat-val {
font-family: 'Bebas Neue', sans-serif;
font-size: 1.8rem;
line-height: 1;
color: var(--gold);
letter-spacing: 0.05em;
}
```

```
.stat-lbl {
  font-size: 0.6rem;
  color: var(--dim);
  letter-spacing: 0.2em;
  text-transform: uppercase;
  margin-top: 4px;
}

/* ——— DECISION BUBBLE ——— */
.decision-bubble {
  grid-column: 1 / -1;
  border: 1px solid var(--border);
  padding: 16px 20px;
  background: var(--panel);
  min-height: 60px;
  font-size: 0.82rem;
  line-height: 1.6;
  color: var(--text);
  font-style: italic;
  position: relative;
}

.decision-bubble::before {
  content: 'AI VERDICT';
  position: absolute;
  top: -9px;
  left: 14px;
  background: var(--panel);
  padding: 0 8px;
  font-style: normal;
  font-family: 'Bebas Neue', sans-serif;
  font-size: 0.7rem;
  letter-spacing: 0.2em;
  color: var(--ai);
  border: 1px solid var(--ai);
}

.flasher {
  animation: flash 0.4s ease;
}

@keyframes flash {
  0% { opacity: 0.2; }
  50% { opacity: 1; }
  100% { opacity: 1; }
}

</style>
</head>
<body>

<h1>Bayesian Confidence Engine</h1>
<p class="subtitle">AI ↔ Human · Bias & Fallacy · Posterior Belief Updates</p>

<div class="engine">

  <!-- BELIEF BARS -->
  <div class="panel belief-section">
    <div class="panel-label">POSTERIOR BELIEF — CURRENT ROUND</div>
    <div class="duel">
      <span class="agent-label human">HUMAN</span>
      <div class="bar-track" style="direction: rtl;">
        <div class="bar-fill human-fill" id="humanBar" style="width: 50%">
          <span class="bar-pct" id="humanPct">50%</span>
        </div>
      </div>
    </div>
    <div class="duel">
      <span class="agent-label ai">AI</span>
      <div class="bar-track">
        <div class="bar-fill ai-fill" id="aiBar" style="width: 50%">
          <span class="bar-pct" id="aiPct">50%</span>
        </div>
      </div>
    </div>
    <div class="prior-display" id="priorDisplay">
      Prior: Human 0.50 · AI 0.50 · Round 0 · No bias detected
    </div>
  </div>

  <!-- SCENARIOS -->
  <div class="panel">
    <div class="panel-label">HUMAN INPUT TYPE</div>
    <div class="scenario-grid" id="scenarioGrid">
      <button class="scenario-btn selected" data-id="0">
        <span class="tag">Anchoring</span>
        States a fact confidently, probably wrong
      </button>
      <button class="scenario-btn" data-id="1">
        <span class="tag">Vague intent</span>
        Asks for "something good" with zero spec
      </button>
      <button class="scenario-btn" data-id="2">
        <span class="tag">Confirmation bias</span>
        Wants validation, not truth
      </button>
      <button class="scenario-btn" data-id="3">
        <span class="tag">Dunning-Kruger</span>
        Certain they're right; demonstrably aren't
      </button>
      <button class="scenario-btn" data-id="4">
        <span class="tag">Framing effect</span>
        Question loaded with implicit assumption
      </button>
    </div>
  </div>
</div>
```

```
<button class="scenario-btn" data-id="5">
  <span class="tag">Clear intent</span>
  Actually knows what they want — rare
</button>
</div>
</div>
```

```
<!-- OUTCOME DISTRIBUTION -->
<div class="panel">
  <div class="panel-label">AI RESPONSE OUTCOME PROBABILITIES</div>
  <div class="outcome-grid" id="outcomeGrid">
    <!-- filled by JS -->
  </div>
</div>
```

```
<!-- AI VERDICT -->
<div class="decision-bubble" id="verdictBubble">
  Select a scenario and fire a round to begin the inference engine.
</div>
```

```
<!-- STATS -->
<div class="stats-row">
  <div class="stat-box">
    <div class="stat-val" id="statRounds">0</div>
    <div class="stat-lbl">Rounds</div>
  </div>
  <div class="stat-box">
    <div class="stat-val" id="statClarify">0</div>
    <div class="stat-lbl">Clarified</div>
  </div>
  <div class="stat-box">
    <div class="stat-val" id="statRight">0</div>
    <div class="stat-lbl">Hit</div>
  </div>
  <div class="stat-box">
    <div class="stat-val" id="statWrong">0</div>
    <div class="stat-lbl">Miss</div>
  </div>
  <div class="stat-box">
    <div class="stat-val" id="statConfidence">—</div>
    <div class="stat-lbl">AI Confidence</div>
  </div>
</div>
```

```
<!-- CONTROLS -->
<div class="controls">
  <button class="btn btn-human" onclick="fireHumanTurn()">► HUMAN SPEAKS</button>
  <button class="btn btn-primary" onclick="fireAITurn()">► AI RESPONDS</button>
  <button class="btn btn-reset" onclick="resetEngine()">↺ RESET PRIORS</button>
</div>
```

```
<!-- LOG -->
<div class="panel log-section">
  <div class="panel-label">INTERACTION LOG</div>
  <div class="log-body" id="logBody"></div>
</div>
```

```
</div>
```

```
<script>
// — State —————
const state = {
  round: 0,
  humanPrior: 0.5, // human's belief they're right
  aiPrior: 0.5,    // AI's belief it can produce the right thing
  selectedScenario: 0,
  counts: { clarify: 0, right: 0, wrong: 0 },
  phase: 'idle', // idle | human_spoke | ai_responded
  lastAIOutcome: null,
  consecutiveCorrect: 0,
  consecutiveFallacy: 0,
};
```

```
// — Scenario definitions —————
const scenarios = [
  {
    id: 0,
    bias: 'Anchoring',
    humanConfidenceDelta: +0.12, // human gets more sure of self
    aiLikelihoodRight: 0.35,
    aiLikelihoodClarify: 0.45,
    aiLikelihoodWrong: 0.20,
    description: 'Human anchors on first claim. AI must decide: challenge the anchor or work around it.',
  },
  {
    id: 1,
    bias: 'Vague Intent',
    humanConfidenceDelta: +0.05,
    aiLikelihoodRight: 0.20,
    aiLikelihoodClarify: 0.60,
    aiLikelihoodWrong: 0.20,
    description: 'Human is vague. AI almost certainly asks for clarification — or guesses and suffers.',
  },
  {
    id: 2,
    bias: 'Confirmation Bias',
    humanConfidenceDelta: +0.18,
    aiLikelihoodRight: 0.25,
    aiLikelihoodClarify: 0.30,
    aiLikelihoodWrong: 0.45,
    description: 'Human wants agreement, not truth. AI producing "truth" will be perceived as wrong.',
  },
  {

```

```
id: 3,
bias: 'Dunning-Kruger',
humanConfidenceDelta: +0.20,
aiLikelihoodRight: 0.30,
aiLikelihoodClarify: 0.35,
aiLikelihoodWrong: 0.35,
description: 'Human is extremely confident yet extremely wrong. High variance outcome for AI',
},
{
id: 4,
bias: 'Framing Effect',
humanConfidenceDelta: +0.08,
aiLikelihoodRight: 0.40,
aiLikelihoodClarify: 0.40,
aiLikelihoodWrong: 0.20,
description: 'Loaded question. AI may detect the frame and reframe — or get pulled inside it.',
},
{
id: 5,
bias: 'Clear Intent',
humanConfidenceDelta: -0.05,
aiLikelihoodRight: 0.75,
aiLikelihoodClarify: 0.15,
aiLikelihoodWrong: 0.10,
description: 'Rare clarity. AI thrives; human is actually humble enough to be specific.',
},
};
```

// — Outcome colours —

```
const outcomes = [
{ key: 'right_exact', label: 'Right (exact match)', color: '#4ae8b0' },
{ key: 'right_partial', label: 'Right (partial / one of many)', color: '#a0e8c0' },
{ key: 'right_lucky', label: 'Right (lucky guess)', color: '#60c8a0' },
{ key: 'clarify', label: 'Asks for clarification', color: '#f0c060' },
{ key: 'wrong_close', label: 'Wrong (close miss)', color: '#e8954a' },
{ key: 'wrong_badly', label: 'Wrong (badly off)', color: '#e85d4a' },
];
```

// — Bayesian update —

```
function bayesUpdate(prior, likelihood, marginal = 0.5) {
const posterior = (likelihood * prior) / marginal;
return Math.min(0.97, Math.max(0.03, posterior));
}
```

```
function clamp(v, lo, hi) { return Math.max(lo, Math.min(hi, v)); }
```

// — Outcome distribution from likelihoods —

```
function computeOutcomeDistribution(sc, round) {
const decay = Math.min(0.3, round × 0.03); // AI gets better over rounds
let r = sc.aiLikelihoodRight + decay;
let c = sc.aiLikelihoodClarify - decay * 0.5;
let w = sc.aiLikelihoodWrong - decay * 0.5;
let total = r + c + w;
r /= total; c /= total; w /= total;
```

// Split right into 3 sub-outcomes, clarify stays, wrong into 2

```
return {
right_exact: r * 0.30,
right_partial: r * 0.55,
right_lucky: r * 0.15,
clarify: c,
wrong_close: w * 0.55,
wrong_badly: w * 0.45,
};
}
```

// — Sample outcome —

```
function sampleOutcome(dist) {
let r = Math.random();
for (const [k, v] of Object.entries(dist)) {
r -= v;
if (r <= 0) return k;
}
return 'wrong_badly';
}
```

// — Render —

```
function render() {
const hp = Math.round(state.humanPrior * 100);
const ap = Math.round(state.aiPrior × 100);
```

```
document.getElementById('humanBar').style.width = hp + '%';
document.getElementById('aiBar').style.width = ap + '%';
document.getElementById('humanPct').textContent = hp + '%';
document.getElementById('aiPct').textContent = ap + '%';
```

```
const sc = scenarios[state.selectedScenario];
document.getElementById('priorDisplay').textContent =
`Prior: Human ${state.humanPrior.toFixed(3)} · AI ${state.aiPrior.toFixed(3)} · Round ${state.round} · ${sc.bias}`;
```

```
const dist = computeOutcomeDistribution(sc, state.round);
const grid = document.getElementById('outcomeGrid');
grid.innerHTML = ``;
for (const o of outcomes) {
const pct = Math.round(dist[o.key] * 100);
const row = document.createElement('div');
row.className = 'outcome-row';
row.innerHTML = `
<span class="outcome-name">${o.label}</span>
<div class="outcome-bar-track">
<div class="outcome-bar-fill" style="width:${pct}%;background:${o.color};"></div>
</div>
<span class="outcome-pct">${pct}%</span>`;
}
```

```
    grid.appendChild(row);
  }

  document.getElementById('statRounds').textContent = state.round;
  document.getElementById('statClarify').textContent = state.counts.clarify;
  document.getElementById('statRight').textContent = state.counts.right;
  document.getElementById('statWrong').textContent = state.counts.wrong;

  const totalDecisions = state×counts×clarify + state×counts×right + state×counts×wrong;
  const confScore = totalDecisions > 0
    ? Math.round((state.counts.right / totalDecisions) * 100) + '%'
    : '—';
  document.getElementById('statConfidence').textContent = confScore;
}

// — Log —————
let tick = 0;
function log(msg, cls = 'ev-sys') {
  tick++;
  const body = document.getElementById('logBody');
  const entry = document.createElement('div');
  entry.className = 'log-entry';
  entry.innerHTML = `<span class="ts">${String(tick).padStart(3, '0')}</span><span class="${cls}">${msg}</span>`;
  body.prepend(entry);
}
```

```
// — Verdict messages —————
const verdicts = {
  right_exact: [
    'Posterior surges. The AI mapped intent precisely — uncommon enough to note.',
    'Clean hit. Human said what they meant; AI believed them. Evidence: both priors rise.',
    'Signal extracted from noise. Exact match achieved.',
  ],
  right_partial: [
    'One of several valid answers produced. Confidence climbs — slowly. There were others.',
    'Partial hit. The truth has many faces; AI found one. Human got what they wanted, not what they needed.',
    'Right — but the distribution of correct answers is wider than the human imagines.',
  ],
  right_lucky: [
    'Lucky draw from the posterior. Do not mistake fortune for calibration.',
    'Correct outcome, questionable path. AI guessed into truth. Entropy will punish this eventually.',
    'Posterior update is modest — the AI knows it got lucky. Or it should.',
  ],
  clarify: [
    'Clarification requested. The AI refuses to hallucinate intent. Epistemically honest; operationally slow.',
    'Inference stalls. AI demands signal before committing. Human interprets this as failure.',
    'Human input was insufficient. AI asks. Human grows impatient. Classic.',
  ],
  wrong_close: [
    'Close miss. The prior was plausible; the human's question was not. Blame distributed.',
    'Wrong — but within variance. The framing trapped the model. Posterior declines.',
    'Adjacent answer. Technically wrong; philosophically understandable.',
  ],
  wrong_badly: [
    'Catastrophic miss. AI produced confidently wrong output. Posterior collapses.',
    'Garbage in, confident garbage out. The human's bias infected the inference.',
    'Posterior bottoms out. A lesson in the compounding cost of unchallenged assumptions.',
  ],
};

function pickVerdict(key) {
  const arr = verdicts[key];
  return arr[Math.floor(Math.random() * arr.length)];
}
```

```
// — Human speaks —————
function fireHumanTurn() {
  const sc = scenarios[state.selectedScenario];
  state.round++;

  // Human's self-confidence rises by scenario delta (with noise)
  const noise = (Math.random() - 0.5) * 0.05;
  state×humanPrior = clamp(state.humanPrior + sc.humanConfidenceDelta + noise, 0.03, 0.97);

  // AI's prior is inverse of human's overconfidence — the more certain the human, the harder the job
  const aiPenalty = sc.humanConfidenceDelta * 0.5;
  state×aiPrior = clamp(state.aiPrior - aiPenalty + noise * 0.5, 0.03, 0.97);

  state×phase = 'human_spoke';
  state×lastAIOutcome = null;

  log(`Human fires: [${sc.bias}] — "${sc.description.split(' ')[0]}", 'ev-human');

  document.getElementById('verdictBubble').textContent =
    `Human has spoken (${sc.bias}). AI is now computing posterior. Fire the AI response to see the outcome.`;
  document.getElementById('verdictBubble').classList.add('flasher');
  setTimeout(() => document.getElementById('verdictBubble').classList.remove('flasher'), 500);

  render();
}
```

```
// — AI responds —————
function fireAITurn() {
  if (state×phase === 'idle') {
    log('AI cannot respond without a human prompt first.', 'ev-warn');
    return;
  }

  const sc = scenarios[state.selectedScenario];
  const dist = computeOutcomeDistribution(sc, state.round);
  const outcome = sampleOutcome(dist);

  const isRight = outcome×startsWith('right');
```

```
const isClarify = outcome === 'clarify';
const isWrong = outcome.startsWith('wrong');

// Update AI prior based on outcome
if (isRight) {
  state.aiPrior = bayesUpdate(state.aiPrior, 0.80);
  state.counts.right++;
  state.consecutiveCorrect++;
  state.consecutiveFallacy = 0;
  log(' AI outcome: ${outcome.toUpperCase().replace('_', ' ')} — posterior rising`, 'ev-ai');
} else if (isClarify) {
  state.aiPrior = clamp(state.aiPrior + 0.02, 0.03, 0.97); // slight reward for epistemic humility
  state.counts.clarify++;
  state.consecutiveFallacy = 0;
  log(' AI outcome: CLARIFICATION REQUESTED — posterior holds`, 'ev-ai');
} else {
  state.aiPrior = bayesUpdate(state.aiPrior, 0.25);
  state.counts.wrong++;
  state.consecutiveCorrect = 0;
  state.consecutiveFallacy++;
  log(' AI outcome: ${outcome.toUpperCase().replace('_', ' ')} — posterior collapses`, 'ev-warn');
}

// Drift human's overconfidence slightly down on wrong answers (they start to doubt)
if (isWrong) {
  state.humanPrior = clamp(state.humanPrior + 0.03, 0.03, 0.97); // they blame the AI — get more certain
}

state.lastAIOutcome = outcome;
state.phase = 'ai_responded';

const bubble = document.getElementById('verdictBubble');
bubble.textContent = pickVerdict(outcome);
bubble.classList.add('flasher');
setTimeout(() => bubble.classList.remove('flasher'), 500);

render();
}

// — Scenario select —
document.getElementById('scenarioGrid').addEventListener('click', e => {
  const btn = e.target.closest('.scenario-btn');
  if (!btn) return;
  document.querySelectorAll('.scenario-btn').forEach(b => b.classList.remove('selected'));
  btn.classList.add('selected');
  state.selectedScenario = parseInt(btn.dataset.id);
  render();
});

// — Reset —
function resetEngine() {
  state.round = 0;
  state.humanPrior = 0.5;
  state.aiPrior = 0.5;
  state.counts = { clarify: 0, right: 0, wrong: 0 };
  state.phase = 'idle';
  state.lastAIOutcome = null;
  state.consecutiveCorrect = 0;
  state.consecutiveFallacy = 0;
  tick = 0;
  document.getElementById('logBody').innerHTML = '';
  document.getElementById('verdictBubble').textContent = 'Priors reset. All beliefs return to 0.50. The universe is indifferent once more.';
  log('Engine reset. Priors uniformed.', 'ev-sys');
  render();
}

// — Init —
log('Bayesian engine initialised. Prior: H=0.500 AI=0.500', 'ev-sys');
render();
</script>
</body>
</html>
```